



Delft University of Technology

Delft University of Technology
Faculty of Electrical Engineering, Mathematics and Computer Science
Department of Electrical and Sustainable Energy
Group of DC Systems, Energy Conversion and Storage

User Manual

Network_builder_public.py
Example_CIGRE_18.py

Author:
Dr. Joel Alpízar-Castillo

Version 1.0
31 July, 2025

Contents

1	Introduction	1
2	csvreader	2
2.1	read_data Class	2
3	Network_builder_public	4
3.1	Create_loads()	5
3.2	Create_admittance_matrix()	5
3.3	Calculate_impedance()	5
3.4	Create_impedance_matrix()	6
3.5	plot_current()	6
3.6	Estimate_Currents()	6
3.7	plot_voltage()	7
3.8	Estimate_Node_voltage()	7
3.9	Create_network()	8
3.10	Network_state()	8
3.11	Create_PV_DF()	8
3.12	BESS_degradation_LFP()	9
3.13	update_BESS_aeging()	9
3.14	Gas_Boiler()	10
3.15	Electric_Power_Balance()	10
3.16	check_P_BESS_setpoint()	11
3.17	house_power_flow()	12
3.18	keep_minTin()	13
3.19	heuristic_control()	13
3.20	GA_control()	15
3.21	Network_Control()	16
3.22	Local_Control()	18
3.23	assign_DF_values()	20
3.24	set_initialConditions()	20
3.25	create_pricesDF()	21
3.26	simulate_Network()	21
3.27	simulate_Network_Centralized_BESS()	22
3.28	Network_Control_Centralized()	23
3.29	Central_BESS_Control()	24
3.30	Voltage_Power_correlation()	25
3.31	Unify_array()	25

3.32	Make_voltage_boxplots()	25
3.33	count_V_compliance()	25
3.34	Plot_grid_behaviour()	26
3.35	case_parameters()	26
3.36	TESS_SoC()	27
3.37	Plot_house_behaviour()	27
3.38	Cost_function_1()	27
3.39	Cost_function_2()	28
3.40	Cost_function()	28
3.41	Find_power_setpoints()	28
4	Example_CIGRE_18	30

Chapter 1

Introduction

This document provides a description of the functions used to simulate the residential multi-carrier energy systems, together with a low-voltage distribution networks in the dissertation [1]. Each chapter provides a description of each of the functions used, including a brief mathematical explanation and details of the arguments. In addition, example files are also provided.

Note that all the functions and example files were tested in Synder 5.5.1, using Python 3.12.7, and Windows 10.

The FLEXINet project was carried out with a Top Sector Energy subsidy from the Ministry of Economic Affairs and Climate, carried out by the Netherlands Enterprise Agency (RVO). The specific subsidy for this project concerns the MOOI subsidy round 2020.

Chapter 2

csvreader

This library was not created for the FLEXINet project, but as part of the MSc thesis [2]. Nevertheless, as it is constantly used, its functions are also detailed. The library itself is stored in the repository in [3].

For this library to be used, the following module must be installed:

- `itertools`
- `pandas`
- `numpy`

2.1 `read_data` Class

The `read_data` class provides a lightweight interface for reading and processing CSV files without relying on full-featured data frames. It allows conversion of CSV data into lists of columns, rows, matrices, or arrays.

Constructor

```
read_data(csv='', delim=';', address='CSVs\\')
```

Arguments:

- **csv [str]:** Name of the CSV file, including extension (e.g., "`data.csv`").
- **delim [str, optional]:** Delimiter used in the CSV file (default is `'`; `'`).
- **address [str, optional]:** Path to the folder containing the CSV file (default is `'CSVs'`).

Attributes:

- **df:** The raw data frame read from the CSV.
- **nrows:** Number of rows in the data.
- **ncols:** Number of columns in the data.

Methods

data2cols()

Converts the data frame into a list of columns. Each column is stored as a list in `self.col`.

data2rows()

Converts the data frame into a list of rows. Each row is stored as a list in `self.row`.

data2matrix()

Converts the data frame into a NumPy matrix and stores it in `self.mat`.

data2array()

Converts the data frame into a NumPy array and stores it in `self.ar`.

Chapter 3

Network_builder_public

This chapter details the functions used to simulate the impact of residential multi-carrier energy systems in low-voltage distribution networks in [4]. Note that, in the paper, confidential data of the Dutch DSO Stedin was used; therefore, the version of the library shared contains only the functions available to the public. Note that most of the functions excluded are related with data handling; therefore, their exclusion does not affect the performance of the library. The library itself is stored in the repository in [5].

For this library to be used, the following modules must be installed:

- pandas
- time
- numpy
- scipy
- math
- re
- matplotlib
- statistics

In addition, the following libraries must be used:

- Probabilistic_Load_Profile_Generator.py
- MCES_library.py
- csvreader.py
- GA.EMS.py

3.1 Create_loads()

Generates nodal power demand profiles for a residential network. The function either loads data from a CSV file or generates synthetic profiles using a probabilistic model. It can also visualize the profiles if requested. The output is a matrix of power demand per node over time, transposed for compatibility with other simulation functions.

Arguments:

- **DF_Network** [**pandas.DataFrame**]: DataFrame containing the network topology and node information.
- **Network_headers** [**list of str**]: List of column headers used to access specific data in the network DataFrame.
- **Load_data** [**bool**]: If True, loads demand profiles from a CSV file; if False, generates synthetic profiles.
- **show_plots** [**bool**]: If True, plots the generated or loaded demand profiles.
- **csv_name** [**str**]: Name of the CSV file containing the demand data.
- **csv_address** [**str**]: Path to the directory containing the CSV file.
- **csv_delim** [**str**]: Delimiter used in the CSV file (e.g., ',' or ';').
- **t** [**int**]: Number of time steps in the simulation (e.g., 365*24*4 for 15-minute resolution over a year).

3.2 Create_admittance_matrix()

Constructs the binary admittance matrix of the network based on the connectivity between nodes. Each entry in the matrix indicates whether a direct connection exists between two nodes. Optionally, the function can display a heatmap of the resulting matrix for visualization purposes.

Arguments:

- **DF_Network** [**pandas.DataFrame**]: DataFrame containing the network topology, including node connections.
- **Network_headers** [**list of str**]: List of column headers used to access node connection data in the network DataFrame.
- **show_heatmap** [**bool**]: If True, displays a heatmap of the admittance matrix using matplotlib.

3.3 Calculate_impedance()

Calculates the complex electrical impedance of a wire segment based on its length and type. The impedance is computed using the resistance and reactance values provided in a wire database. The result is returned either in ohms or ohms per kilometer depending on the input flag.

Arguments:

- **DF_Wires** [**pandas.DataFrame**]: DataFrame containing wire specifications including resistance and reactance.

- **distance [float]:** Length of the wire segment in meters or kilometers.
- **wire_label [str]:** Identifier for the wire type used to look up impedance values.
- **Wires_headers [list of str]:** List of column headers used to access wire data in the DataFrame.
- **km [bool]:** If True, the distance is interpreted in kilometers and impedance is scaled accordingly; otherwise, distance is in meters.

3.4 Create_impedance_matrix()

Generates the complex impedance matrix of the network using wire specifications and network topology. Each entry in the matrix represents the impedance between two connected nodes, calculated from wire resistance and reactance. Optionally, the function can display a heatmap of the impedance magnitudes.

Arguments:

- **DF_Network [pandas.DataFrame]:** DataFrame containing the network topology, including node connections and wire labels.
- **Network_headers [list of str]:** List of column headers used to access node and wire data in the network DataFrame.
- **Wires_headers [list of str]:** List of column headers used to access wire specifications in the wire DataFrame.
- **show_heatmap [bool]:** If True, displays a heatmap of the impedance matrix using matplotlib.
- **wire_file [str]:** Path to the Excel file containing wire specifications.
- **km [bool]:** If True, wire distances are interpreted in kilometers; otherwise, in meters.

3.5 plot_current()

Plots the current vector across network nodes. The function visualizes the magnitude and direction of the current at each node, helping to assess the distribution and flow of electrical current in the network.

Arguments:

- **I [numpy.ndarray]:** Array of complex current values per node, where each entry represents the current magnitude and phase at a specific node.

3.6 Estimate_Currents()

Estimates the current vector in a low-voltage distribution network using nodal power and voltage values. The function iteratively solves the current balance equations until convergence is achieved within a specified tolerance. Optionally, it can visualize the resulting current distribution.

Arguments:

- **A [numpy.ndarray]:** Admittance matrix representing node connectivity.

- **S_n [numpy.ndarray]:** Array of complex nodal power values (in kW).
- **I_0 [numpy.ndarray]:** Initial guess for the current vector.
- **V_0 [numpy.ndarray]:** Voltage vector at each node (in volts).
- **dI [float]:** Convergence threshold for current iteration (default is 0.01 A).
- **show_plot [bool]:** If True, plots the resulting current vector.

3.7 plot_voltage()

Plots the voltage vector across network nodes in per-unit (pu) values. This visualization helps assess voltage levels and deviations throughout the network, which is critical for evaluating voltage compliance and quality of service.

Arguments:

- **V_n [numpy.ndarray]:** Array of complex voltage values at each node.
- **V_feeder [float]:** Reference voltage of the feeder (default is 400 V), used to normalize the voltage values to per-unit.

3.8 Estimate_Node_voltage()

Estimates the voltage at each node in a low-voltage distribution network using the admittance and impedance matrices, nodal power demand, and feeder voltage. The function iteratively solves the voltage balance equations until convergence is achieved within a specified tolerance. It also updates the current vector accordingly and can visualize the voltage profile.

Arguments:

- **A [numpy.ndarray]:** Admittance matrix representing node connectivity.
- **Z [numpy.ndarray]:** Impedance matrix representing the electrical characteristics between nodes.
- **S_n [numpy.ndarray]:** Array of complex nodal power values (in kW).
- **I_0 [numpy.ndarray]:** Initial guess for the current vector.
- **V_0 [numpy.ndarray]:** Initial guess for the voltage vector (in volts).
- **V_feeder [float]:** Reference voltage of the feeder (default is 400 V).
- **dV [float]:** Convergence threshold for voltage iteration (default is 0.001 V).
- **dI [float]:** Convergence threshold for current iteration (default is 0.01 A).
- **show_plot [bool]:** If True, plots the resulting voltage vector.

3.9 Create_network()

Creates the admittance and impedance matrices for a low-voltage distribution network. These matrices are essential for power flow analysis and voltage estimation. The function uses the network topology and wire specifications to compute the matrices.

Arguments:

- **DF_Network** [**pandas.DataFrame**]: DataFrame containing the network topology, including node connections and wire labels.
- **Network_headers** [**list of str**]: List of column headers used to access node and wire data in the network DataFrame.
- **Wires_headers** [**list of str**]: List of column headers used to access wire specifications in the wire DataFrame.
- **wire_file** [**str**]: Path to the Excel file containing wire specifications (default is 'Gaia_cables.xlsx').

3.10 Network_state()

Computes the voltage and current state of the network at a given timestep using nodal power data. It uses the admittance and impedance matrices to estimate the voltage at each node and the corresponding current vector. This function is essential for time-series simulation of network behavior.

Arguments:

- **A** [**numpy.ndarray**]: Admittance matrix representing node connectivity.
- **Z** [**numpy.ndarray**]: Impedance matrix representing the electrical characteristics between nodes.
- **df_S_n** [**pandas.DataFrame**]: DataFrame containing time-series nodal power demand data (in kW).
- **timestep** [**int**]: Index of the time step to evaluate.
- **I_0** [**numpy.ndarray**]: Initial guess for the current vector.
- **V_0** [**numpy.ndarray**]: Initial guess for the voltage vector (in volts).

3.11 Create_PV_DF()

Generates a DataFrame containing photovoltaic (PV) power profiles for each node in the network. The profiles are either scaled from reference values or estimated based on the nodal energy demand. The function can also return the number of PV modules required per node.

Arguments:

- **S_n** [**numpy.ndarray or bool**]: Matrix of nodal power demand profiles. If False, PV sizing is based on reference values.
- **t_0** [**int**]: Starting timestep for the simulation (default is 0).

- **t_simulation [int]:** Number of timesteps to simulate (default is 96, i.e., one day at 15-minute resolution).
- **H [int]:** Forecast horizon extension in timesteps (default is 0).
- **P_PV_ref [numpy.ndarray]:** Reference PV power values per node (in kW).
- **module_power_ref [float]:** Rated power of a single PV module (default is 0.315 kW).
- **return_modules [bool]:** If True, returns the number of modules per node instead of the power profile.
- **dt [float]:** Time resolution of the simulation in hours (default is 0.25 h).

3.12 BESS_degradation_LFP()

Models the degradation of a lithium iron phosphate (LFP) battery based on its state of charge (SoC), temperature, voltage, and usage pattern. The function supports multiple degradation models (e.g., Olmos, Vermeer, Wang) and returns the updated battery capacity after accounting for both cyclic and calendar aging effects.

Arguments:

- **SoC_0 [float]:** Initial state of charge of the battery (0 to 1).
- **SoC_f [float]:** Final state of charge after the current cycle (0 to 1).
- **Capacity_BESS [float]:** Current usable capacity of the battery (in kWh).
- **Capacity_BESS_BOL [float]:** Beginning-of-life capacity of the battery (default is 10 kWh).
- **T [float]:** Operating temperature in kelvin (default is 293 K).
- **V [float]:** Nominal voltage of the battery (default is 62.7 V).
- **model [str]:** Degradation model to use ('Wang', 'Olmos', or 'Vermeer').
- **dt [float]:** Time step duration in hours (default is 0.25 h).

3.13 update_BESS_aeaging()

Updates the state of charge (SoC) and capacity of a battery energy storage system (BESS) based on its power usage and degradation model. The function accounts for charge/discharge efficiency and applies the Wang degradation model to estimate capacity loss over time.

Arguments:

- **SoC_0 [float]:** Initial state of charge of the battery (0 to 1).
- **P_BESS [float]:** Power exchanged with the battery (positive for discharge, negative for charge) in kW.
- **Capacity_BESS [float]:** Current usable capacity of the battery in kWh.

- **Capacity_BESS_BoL [float]:** Beginning-of-life capacity of the battery (default is 10 kWh).
- **charge_efficiency [float]:** Efficiency factor for charging the battery (default is 0.943).
- **discharge_efficiency [float]:** Efficiency factor for discharging the battery (default is 0.943).
- **dt [float]:** Time step duration in hours (default is 0.25 h).

3.14 Gas_Boiler()

Calculates the thermal power output of a gas boiler and updates the indoor temperature of a residential building based on its thermal characteristics. The function considers the heat demand, ambient conditions, and building envelope properties to determine whether the boiler should operate and how much heat it should supply.

Arguments:

- **T_in [float]:** Current indoor temperature in kelvin.
- **T_amb [float]:** Ambient (outdoor) temperature in kelvin.
- **T_set [float]:** Desired indoor setpoint temperature in kelvin.
- **mdot [float]:** Mass flow rate of the heating fluid in kg/s (default is 0.1).
- **T_network [float]:** Temperature of the heating network supply in kelvin (default is 323 K).
- **efficiency [float]:** Efficiency of the gas boiler (default is 0.8).
- **c [float]:** Specific heat capacity of the heating fluid in J/(kg·K) (default is 4200).
- **House_type [str]:** Type of house (e.g., 'apartment', 'detached').
- **glazing_type [str]:** Type of window glazing (e.g., 'single', 'double').
- **Cavity_Filling [bool]:** Indicates whether the wall cavity is filled with insulation.
- **Cavity_type [str]:** Type of cavity insulation material (e.g., 'air', 'foam').
- **Wall_Cover [bool]:** Indicates whether the walls have additional external covering.
- **LRoof [float]:** Thickness of the roof insulation layer in meters.
- **LCavity_wall [float]:** Thickness of the cavity wall insulation in meters.

3.15 Electric_Power_Balance()

Balances the electric power among household components—load, heat pump (HP), battery energy storage system (BESS), photovoltaic (PV) generation, and grid exchange—based on operational constraints and grid setpoints. The function determines how much power should be supplied or curtailed by each component to meet the grid operator's requirements.

Arguments:

- **P_Load [float]:** Power demand of the household load in kW.
- **P_HP [float]:** Power consumption of the heat pump in kW.
- **P_BESS [float]:** Requested power from the battery system in kW.
- **P_BESS_max [float]:** Maximum allowable discharge power from the BESS in kW.
- **P_BESS_min [float]:** Minimum allowable charge power to the BESS in kW.
- **P_PV [float]:** Available photovoltaic power generation in kW.
- **P_grid_DSO [float]:** Power setpoint requested by the distribution system operator (DSO) in kW.
- **top [bool]:** If True, enables peak shaving mode; otherwise, curtailment mode.
- **exact_match [bool]:** If True, enforces strict matching of the grid setpoint; otherwise, allows partial compensation.

3.16 check_P_BESS_setpoint()

Checks whether the battery energy storage system (BESS) can meet the power setpoint required to satisfy the household's net power balance and the grid operator's request. It evaluates the feasibility of the setpoint within the operational limits of the BESS and returns the adjusted setpoint, the curtailed PV power (if any), and a flag indicating whether the setpoint is feasible. Arguments:

- **SoC_0 [float]:** Initial state of charge of the battery (0 to 1).
- **Capacity_BESS [float]:** Current usable capacity of the battery in kWh.
- **P_Load [float]:** Household load power demand in kW.
- **P_HP [float]:** Power consumption of the heat pump in kW.
- **P_HP_TESS [float]:** Power used by the heat pump to charge the thermal energy storage system (TESS) in kW.
- **P_PV_av [float]:** Available photovoltaic power generation in kW.
- **P_grid_DSO [float]:** Power setpoint requested by the distribution system operator (DSO) in kW.
- **P_BESS_max [float]:** Maximum allowable power exchange with the BESS in kW (default is 10).
- **dt [float]:** Time step duration in hours (default is 0.25 h).

3.17 house_power_flow()

Simulates the internal power flow of a residential household, accounting for interactions between the load, heat pump (HP), battery energy storage system (BESS), photovoltaic (PV) generation, and thermal energy storage system (TESS). The function calculates the power exchanged with the grid and updates the thermal and electrical states of the house.

Arguments:

- **enable_TESS** [bool]: Enables the use of the thermal energy storage system.
- **enable_HP** [bool]: Enables the use of the heat pump.
- **TESS_active** [bool]: Indicates whether the TESS is currently active.
- **HP_active** [bool]: Indicates whether the heat pump is currently active.
- **HP_charge_TESS** [bool]: Indicates whether the heat pump is charging the TESS.
- **T_amb** [float]: Ambient temperature in kelvin.
- **SoC_0** [float]: Initial state of charge of the battery (0 to 1).
- **P_Load** [float]: Household electrical load in kW.
- **P_PV_av** [float]: Available photovoltaic power in kW.
- **T_TESS_0** [float]: Initial temperature of the TESS in kelvin.
- **T_ret** [float]: Return temperature of the heating system in kelvin.
- **T_soil** [float]: Soil temperature in kelvin.
- **Capacity_BESS** [float]: Current usable capacity of the battery in kWh.
- **P_grid_DSO** [float or bool]: Grid power setpoint requested by the DSO (optional).
- **top** [bool]: If True, enables peak shaving mode; otherwise, curtailment mode.
- **P_BESS_ref** [float]: Reference power setpoint for the battery in kW.
- **Capacity_BESS_BoL** [float]: Beginning-of-life capacity of the battery in kWh.
- **P_BESS_max** [float]: Maximum allowable power exchange with the battery in kW.
- **T_TESS_max** [float]: Maximum allowable temperature of the TESS in kelvin.
- **setpoint** [bool]: If True, uses the provided battery setpoint; otherwise, computes it internally.

3.18 keep_minTin()

Ensures that the indoor temperature of a household does not fall below a minimum threshold. If the current indoor temperature is below the specified minimum, the function activates the heat pump and disables the thermal energy storage system (TESS) charging. Otherwise, it retains the current operational states of the devices.

Arguments:

- **T_in [float]:** Current indoor temperature in kelvin.
- **TESS_active [bool]:** Current activation state of the thermal energy storage system.
- **HP_active [bool]:** Current activation state of the heat pump.
- **HP_charge_TESS [bool]:** Current state indicating whether the heat pump is charging the TESS.
- **T_in_min [float]:** Minimum allowable indoor temperature in kelvin (default is 289 K, i.e., 16°C).

3.19 heuristic_control()

Implements a rule-based (heuristic) control strategy for managing household energy systems, including heat pumps (HP), thermal energy storage systems (TESS), and battery energy storage systems (BESS). The control logic is based on indoor temperature, electricity prices, and grid constraints. It determines the operational states of devices and computes the resulting power flows and thermal dynamics.

Arguments:

- **T_in [float]:** Current indoor temperature in kelvin.
- **T_set [float]:** Desired indoor temperature setpoint in kelvin.
- **T_amb [float]:** Ambient (outdoor) temperature in kelvin.
- **T_TESS_0 [float]:** Initial temperature of the thermal energy storage system in kelvin.
- **T_ret [float]:** Return temperature of the heating system in kelvin.
- **T_soil [float]:** Soil temperature profile in kelvin.
- **G [float]:** Solar irradiance in W/m².
- **P_Load [float]:** Household electrical load in kW.
- **P_grid_DSO [float]:** Power setpoint requested by the distribution system operator in kW.
- **top [bool]:** If True, enables peak shaving mode; otherwise, curtailment mode.
- **SoC_0 [float]:** Initial state of charge of the battery (0 to 1).
- **Cost_grid [float]:** Current electricity price in €/kWh.
- **Cost_grid_median [float]:** Median electricity price for the day in €/kWh.

- **Cost_grid_Q1 [float]:** First quartile of electricity prices in €/kWh.
- **Cost_grid_Q3 [float]:** Third quartile of electricity prices in €/kWh.
- **Capacity_BESS [float]:** Current usable capacity of the battery in kWh.
- **P_PV_av [float]:** Available photovoltaic power in kW.
- **P_BESS_max [float]:** Maximum allowable power exchange with the battery in kW (default is 10).
- **Capacity_BESS_BoL [float]:** Beginning-of-life capacity of the battery in kWh (default is 10).
- **enable_HP [bool]:** Enables the use of the heat pump.
- **enable_TESS [bool]:** Enables the use of the thermal energy storage system.
- **enable_HP_2_TESS [bool]:** Enables the heat pump to charge the TESS.
- **external_control [bool]:** If True, uses external control logic; otherwise, uses internal heuristic.
- **SoC_BESS_min [float]:** Minimum allowable state of charge for the battery (default is 0.9).
- **SoC_BESS_max [float]:** Maximum allowable state of charge for the battery (default is 0.2).
- **m [float]:** Thermal mass of the building in kg (default is 4000).
- **TESS_min_op_T [float]:** Minimum operational temperature of the TESS in kelvin (default is 328 K).
- **TESS_max_op_T [float]:** Maximum operational temperature of the TESS in kelvin (default is 363 K).
- **T_TESS_min [float]:** Minimum temperature of the TESS in kelvin (default is 323 K).
- **T_TESS_max [float]:** Maximum temperature of the TESS in kelvin (default is 363 K).
- **T_in_crit [float]:** Critical indoor temperature threshold in kelvin (default is 289 K).
- **m_dot [float]:** Mass flow rate of the heating fluid in kg/s (default is 0.22).
- **c_f [float]:** Specific heat capacity of the heating fluid in J/(kg·K) (default is 4200).
- **dt [float]:** Time step duration in hours (default is 0.25 h).
- **House_type [str]:** Type of house (e.g., 'apartment', 'detached').
- **force_temperature [bool]:** If True, forces the indoor temperature to match the setpoint.
- **glazing_type [str]:** Type of window glazing (e.g., 'single', 'double').
- **Cavity_Filling [bool]:** Indicates whether the wall cavity is filled with insulation.
- **Cavity_type [str]:** Type of cavity insulation material (e.g., 'air', 'foam').

- **Wall_Cover [bool]:** Indicates whether the walls have additional external covering.
- **LRoof [float]:** Thickness of the roof insulation layer in meters.
- **LCavity_wall [float]:** Thickness of the cavity wall insulation in meters.

3.20 GA_control()

Uses a genetic algorithm (GA) to optimize the operation of a household energy system over a forecast horizon. The optimization considers electricity prices, thermal comfort, and energy storage constraints to determine the best control actions for the heat pump (HP), thermal energy storage system (TESS), and battery energy storage system (BESS).

Arguments:

- **T_in [float]:** Current indoor temperature in kelvin.
- **T_set [list of float]:** Desired indoor temperature setpoints over the forecast horizon in kelvin.
- **T_amb [list of float]:** Ambient temperature forecast over the horizon in kelvin.
- **T_TESS_0 [float]:** Initial temperature of the TESS in kelvin.
- **T_ret [float]:** Return temperature of the heating system in kelvin.
- **T_soil [list of float]:** Soil temperature forecast over the horizon in kelvin.
- **G [list of float]:** Solar irradiance forecast over the horizon in W/m^2 .
- **P_Load [list of float]:** Household electrical load forecast over the horizon in kW.
- **P_grid_DSO [float]:** Grid power setpoint requested by the DSO in kW.
- **top [bool]:** If True, enables peak shaving mode; otherwise, curtailment mode.
- **SoC_0 [float]:** Initial state of charge of the battery (0 to 1).
- **costs [list of list]:** Forecasted energy and CO costs over the horizon.
- **Capacity_BESS [float]:** Current usable capacity of the battery in kWh.
- **P_PV_av [list of float]:** Forecasted available photovoltaic power over the horizon in kW.
- **horizon [int]:** Number of timesteps in the forecast horizon (default is 0).
- **P_BESS_max [float]:** Maximum allowable power exchange with the battery in kW (default is 10).
- **Capacity_BESS_BoL [float]:** Beginning-of-life capacity of the battery in kWh (default is 10).
- **enable_HP_2_TESS [bool]:** Enables the heat pump to charge the TESS (default is True).
- **external_control [bool]:** If True, uses external control logic (default is False).
- **SoC_BESS_min [float]:** Minimum allowable state of charge for the battery (default is 0.9).

- **SoC_BESS_max** [float]: Maximum allowable state of charge for the battery (default is 0.2).
- **m** [float]: Thermal mass of the building in kg (default is 4000).
- **TESS_min_op_T** [float]: Minimum operational temperature of the TESS in kelvin (default is 328 K).
- **TESS_max_op_T** [float]: Maximum operational temperature of the TESS in kelvin (default is 363 K).
- **T_TESS_min** [float]: Minimum temperature of the TESS in kelvin (default is 323 K).
- **T_TESS_max** [float]: Maximum temperature of the TESS in kelvin (default is 363 K).
- **m_dot** [float]: Mass flow rate of the heating fluid in kg/s (default is 0.22).
- **c_f** [float]: Specific heat capacity of the heating fluid in J/(kg·K) (default is 4200).
- **dt** [float]: Time step duration in hours (default is 0.25 h).
- **House_type** [str]: Type of house (e.g., 'apartment').

3.21 Network_Control()

Coordinates the control of distributed energy resources across a low-voltage network. It applies local control strategies to selected and remaining nodes, evaluates voltage compliance, and adjusts power setpoints if necessary. The function ensures that grid constraints are respected and updates the operational states of all nodes.

Arguments:

- **A** [numpy.ndarray]: Admittance matrix representing node connectivity.
- **Z** [numpy.ndarray]: Impedance matrix representing electrical characteristics between nodes.
- **I_0** [numpy.ndarray]: Initial current vector for the network.
- **V_0** [numpy.ndarray]: Initial voltage vector for the network.
- **Network_headers** [list of str]: Column headers used to access network topology and metadata.
- **selection** [list of int]: List of node indices selected for active control.
- **remaining** [list of int]: List of node indices not selected for active control.
- **df_P_PV_av** [pandas.DataFrame]: Available photovoltaic power per node over time.
- **df_P_PV** [pandas.DataFrame]: Actual photovoltaic power injected per node.
- **df_P_L** [pandas.DataFrame]: Household load power per node over time.
- **df_P_HP** [pandas.DataFrame]: Heat pump power consumption per node.
- **df_P_HP_TESS** [pandas.DataFrame]: Power used by heat pumps to charge TESS.

- **df_P_BESS** [pandas.DataFrame]: Battery power exchange per node.
- **df_P_Grid** [pandas.DataFrame]: Net grid power per node.
- **df_P_ref** [pandas.DataFrame]: Grid power setpoints requested by the DSO.
- **df_DSO_operation** [pandas.DataFrame]: Flags indicating whether DSO control is active per node.
- **df_SOC_BESS** [pandas.DataFrame]: Battery state of charge per node.
- **df_C_BESS** [pandas.DataFrame]: Battery capacity per node.
- **df_Qdot_D** [pandas.DataFrame]: Thermal demand per node.
- **df_Qdot_HP** [pandas.DataFrame]: Thermal output from heat pumps per node.
- **df_Qdot_HP_TESS** [pandas.DataFrame]: Thermal energy used to charge TESS per node.
- **df_Qdot_TESS** [pandas.DataFrame]: Thermal output from TESS per node.
- **df_enable_HP_2_TESS** [pandas.DataFrame]: Flags indicating whether HP-to-TESS charging is enabled.
- **df_T_TESS** [pandas.DataFrame]: Temperature of TESS per node.
- **df_Qdot_TESS_SD** [pandas.DataFrame]: Thermal losses from TESS to soil.
- **df_Qdot_Boiler** [pandas.DataFrame]: Thermal output from gas boilers per node.
- **df_T_in** [pandas.DataFrame]: Indoor temperature per node.
- **df_Prices** [pandas.DataFrame]: Electricity price data including current and statistical summaries.
- **T_amb** [list of float]: Ambient temperature profile over time.
- **G** [list of float]: Solar irradiance profile over time.
- **T_set** [list of float]: Indoor temperature setpoints over time.
- **T_soil** [list of list]: Soil temperature profile per depth layer.
- **df_House_type** [pandas.DataFrame]: Mapping of node indices to house types.
- **timestep** [int]: Current simulation timestep.
- **control_type** [str]: Control strategy to apply ('heuristic' or 'GA').
- **follow_control** [bool]: If True, enforces external control setpoints.
- **P_BESS_max** [float]: Maximum allowable power exchange with the battery.
- **Capacity_BESS_BoL** [float]: Beginning-of-life battery capacity.
- **enable_HP** [bool]: Enables heat pump operation.
- **enable_TESS** [bool]: Enables TESS operation.

- **V_ref_min** [float]: Minimum allowable voltage threshold (default is 0.95×400 V).
- **V_ref_max** [float]: Maximum allowable voltage threshold (default is 1.05×400 V).
- **P_ref_min** [float]: Minimum allowable grid power setpoint (default is -1.48 kW).
- **P_ref_max** [float]: Maximum allowable grid power setpoint (default is 1.38 kW).
- **force_temperature** [bool]: If True, forces indoor temperature to match setpoint.

3.22 Local_Control()

Applies local control strategies to individual nodes in a low-voltage distribution network. Depending on the selected control type (heuristic or genetic algorithm), the function determines the operation of household energy systems such as heat pumps (HP), thermal energy storage systems (TESS), and battery energy storage systems (BESS). It updates power flows, thermal states, and control flags for each node.

Arguments:

- **Network_headers** [list of str]: Column headers used to access network topology and metadata.
- **selection** [list of int]: List of node indices selected for active control.
- **remaining** [list of int]: List of node indices not selected for active control.
- **df_P_PV_av** [pandas.DataFrame]: Available photovoltaic power per node over time.
- **df_P_PV** [pandas.DataFrame]: Actual photovoltaic power injected per node.
- **df_P_L** [pandas.DataFrame]: Household load power per node over time.
- **df_P_HP** [pandas.DataFrame]: Heat pump power consumption per node.
- **df_P_HP_TESS** [pandas.DataFrame]: Power used by heat pumps to charge TESS.
- **df_P_BESS** [pandas.DataFrame]: Battery power exchange per node.
- **df_P_Grid** [pandas.DataFrame]: Net grid power per node.
- **df_P_ref** [pandas.DataFrame]: Grid power setpoints requested by the DSO.
- **df_DSO_operation** [pandas.DataFrame]: Flags indicating whether DSO control is active per node.
- **df_SOC_BESS** [pandas.DataFrame]: Battery state of charge per node.
- **df_C_BESS** [pandas.DataFrame]: Battery capacity per node.
- **df_Qdot_D** [pandas.DataFrame]: Thermal demand per node.
- **df_Qdot_HP** [pandas.DataFrame]: Thermal output from heat pumps per node.
- **df_Qdot_HP_TESS** [pandas.DataFrame]: Thermal energy used to charge TESS per node.

- **df_Qdot_TESS** [**pandas.DataFrame**]: Thermal output from TESS per node.
- **df_enable_HP_2_TESS** [**pandas.DataFrame**]: Flags indicating whether HP-to-TESS charging is enabled.
- **df_T_TESS** [**pandas.DataFrame**]: Temperature of TESS per node.
- **df_Qdot_TESS_SD** [**pandas.DataFrame**]: Thermal losses from TESS to soil.
- **df_Qdot_Boiler** [**pandas.DataFrame**]: Thermal output from gas boilers per node.
- **df_T_in** [**pandas.DataFrame**]: Indoor temperature per node.
- **df_Prices** [**pandas.DataFrame**]: Electricity price data including current and statistical summaries.
- **T_amb** [**list of float**]: Ambient temperature profile over time.
- **G** [**list of float**]: Solar irradiance profile over time.
- **T_set** [**list of float**]: Indoor temperature setpoints over time.
- **T_soil** [**list of list**]: Soil temperature profile per depth layer.
- **df_House_type** [**pandas.DataFrame**]: Mapping of node indices to house types.
- **ts** [**int**]: Current simulation timestep.
- **control_type** [**str**]: Control strategy to apply ('heuristic' or 'GA').
- **follow_control** [**bool**]: If True, enforces external control setpoints.
- **horizon** [**int**]: Forecast horizon for GA control (default is 0).
- **energy_costs** [**list of float**]: Energy cost coefficients for optimization.
- **CO2_costs** [**list of float**]: CO2 cost coefficients for optimization.
- **T_ret** [**float**]: Return temperature of the heating system in kelvin (default is 311 K).
- **method** [**bool**]: Forecasting method for GA control (default is False).
- **P_BEES_max** [**float**]: Maximum allowable power exchange with the battery.
- **Capacity_BEES_BoL** [**float**]: Beginning-of-life battery capacity.
- **enable_HP** [**bool**]: Enables heat pump operation.
- **enable_TESS** [**bool**]: Enables TESS operation.
- **force_temperature** [**bool**]: If True, forces indoor temperature to match setpoint.
- **glazing_type** [**str**]: Type of window glazing (e.g., 'double').
- **Cavity_Filling** [**bool**]: Indicates whether the wall cavity is filled with insulation.
- **Cavity_type** [**str**]: Type of cavity insulation material (e.g., 'air').
- **Wall_Cover** [**bool**]: Indicates whether the walls have additional external covering.
- **LRoof** [**float**]: Thickness of the roof insulation layer in meters.
- **LCavity_wall** [**float**]: Thickness of the cavity wall insulation in meters.

3.23 assign_DF_values()

Assigns a specified value to selected columns in a given row of a pandas DataFrame. This function is typically used to initialize or update simulation variables for specific nodes in the network.

Arguments:

- **DF** [pandas.DataFrame]: The DataFrame to be modified.
- **row** [int]: Index of the row in the DataFrame where values will be assigned.
- **columns** [list of int]: List of column indices to which the value will be assigned.
- **value** [any]: Value to assign to the specified columns (default is 0).

3.24 set_initialConditions()

Initializes the state variables of the network simulation, including power flows, battery and thermal storage states, and indoor temperatures. This function assigns default or user-defined values to selected and remaining nodes at the beginning of the simulation.

Arguments:

- **Network_headers** [list of str]: Column headers used to access network topology and metadata.
- **selection** [list of int]: List of node indices selected for active control.
- **remaining** [list of int]: List of node indices not selected for active control.
- **df_P_PV** [pandas.DataFrame]: DataFrame for photovoltaic power per node.
- **df_P_L** [pandas.DataFrame]: DataFrame for household load power per node.
- **df_P_HP** [pandas.DataFrame]: DataFrame for heat pump power per node.
- **df_P_HP_TESS** [pandas.DataFrame]: DataFrame for power used to charge TESS.
- **df_P_BEES** [pandas.DataFrame]: DataFrame for battery power per node.
- **df_P_Grid** [pandas.DataFrame]: DataFrame for net grid power per node.
- **df_P_ref** [pandas.DataFrame]: DataFrame for DSO power setpoints per node.
- **df_DSO_operation** [pandas.DataFrame]: DataFrame indicating DSO control status per node.
- **df_SOC_BEES** [pandas.DataFrame]: DataFrame for battery state of charge per node.
- **df_C_BEES** [pandas.DataFrame]: DataFrame for battery capacity per node.
- **df_Qdot_D** [pandas.DataFrame]: DataFrame for thermal demand per node.
- **df_Qdot_HP** [pandas.DataFrame]: DataFrame for thermal output from heat pumps.
- **df_Qdot_HP_TESS** [pandas.DataFrame]: DataFrame for thermal energy used to charge TESS.

- **df_Qdot_TESS** [**pandas.DataFrame**]: DataFrame for thermal output from TESS.
- **df_enable_HP_2_TESS** [**pandas.DataFrame**]: DataFrame indicating HP-to-TESS charging status.
- **df_T_TESS** [**pandas.DataFrame**]: DataFrame for TESS temperature per node.
- **df_Qdot_TESS_SD** [**pandas.DataFrame**]: DataFrame for thermal losses from TESS to soil.
- **df_T_in** [**pandas.DataFrame**]: DataFrame for indoor temperature per node.
- **T_amb** [**list of float**]: Ambient temperature profile over time.
- **T_soil** [**list of list**]: Soil temperature profile per depth layer.
- **df_House_type** [**pandas.DataFrame**]: Mapping of node indices to house types.
- **initial_conditions** [**list**]: List of default values for initializing state variables (optional).

3.25 create_pricesDF()

This function generates a DataFrame containing statistical summaries of energy prices over a simulation period. It calculates daily minimum, 25th percentile (Q25), median, current, 75th percentile (Q75), and maximum values for each time step.

Arguments:

- **Energy_price** [**list of floats**]: List of energy prices over time, typically in €/kWh.
- **t_simulation** [**int**]: Total number of time steps in the simulation.
- **t_0** [**int, optional**]: Initial time step index. Default is 0.
- **dt** [**float, optional**]: Time resolution in hours. Default is 0.25 (i.e., 15 minutes).

3.26 simulate_Network()

This function simulates the behavior of a low-voltage residential network over a specified time period. It models the electrical and thermal dynamics of the network, including power flows, voltage profiles, and interactions with distributed energy resources such as PV systems, batteries (BESS), and thermal energy storage systems (TESS).

Arguments:

- **DF_Network** [**pandas.DataFrame**]: Network topology and node metadata.
- **Network_headers** [**list**]: Column headers for the network DataFrame.
- **Wires_headers** [**list**]: Column headers for the wire specification file.
- **selection** [**list**]: List of node indices with active control.
- **remaining** [**list**]: List of node indices without active control.
- **S_n** [**array**]: Power demand matrix for each node over time.

- **t_simulation** [int]: Total number of time steps to simulate.
- **V_0** [array, optional]: Initial voltage vector. Default is feeder voltage.
- **I_0** [array, optional]: Initial current vector. Default is zero.
- **t_0** [int, optional]: Starting time index. Default is 0.
- **wire_file** [str, optional]: Filename of the wire specification Excel file. Default is 'Wires.xlsx'.
- **H** [int, optional]: Forecast horizon. Default is 0.
- **control_type** [str, optional]: Type of control strategy ('heuristic' or 'GA'). Default is 'heuristic'.
- **follow_control** [bool, optional]: Whether to follow external control signals. Default is False.
- **P_BESS_max** [float, optional]: Maximum battery power per node. Default is 10 kW.
- **Capacity_BESS_BoL** [float, optional]: Initial battery capacity per node. Default is 10 kWh.
- **enable_HP** [bool, optional]: Enable heat pump operation. Default is True.
- **enable_TESS** [bool, optional]: Enable thermal energy storage system. Default is True.
- **V_feeder** [float, optional]: Feeder voltage level. Default is 400 V.
- **dt** [float, optional]: Time resolution in hours. Default is 0.25.
- **force_temperature** [bool, optional]: Force indoor temperature to match setpoint. Default is False.
- **start_day** [int, optional]: Start day of simulation. Default is 0.
- **end_day** [int, optional]: End day of simulation. Default is 1.

3.27 simulate_Network_Centralized_BESS()

This function simulates a low-voltage residential network with a centralized battery energy storage system (BESS). It evaluates the network's voltage and current profiles over time, considering centralized control strategies for charging and discharging the BESS to support grid stability.

Arguments:

- **DF_Network** [pandas.DataFrame]: Network topology and node metadata.
- **Network_headers** [list]: Column headers for the network DataFrame.
- **Wires_headers** [list]: Column headers for the wire specification file.
- **selection** [list]: List of node indices with centralized BESS.
- **remaining** [list]: List of node indices without BESS.
- **S_n** [array]: Power demand matrix for each node over time.

- **P_PV_peak** [array]: Peak PV generation per node.
- **df_P_L** [pandas.DataFrame]: Load profile per node.
- **t_simulation** [int]: Total number of time steps to simulate.
- **V_0** [array, optional]: Initial voltage vector. Default is feeder voltage.
- **I_0** [array, optional]: Initial current vector. Default is zero.
- **t_0** [int, optional]: Starting time index. Default is 0.
- **wire_file** [str, optional]: Filename of the wire specification Excel file. Default is 'Wires.xlsx'.
- **H** [int, optional]: Forecast horizon. Default is 0.
- **control_type** [str, optional]: Type of control strategy. Default is 'heuristic'.
- **follow_control** [bool, optional]: Whether to follow external control signals. Default is False.
- **P_BESS_max** [float, optional]: Maximum power capacity of the centralized BESS. Default is 1000 kW.
- **Capacity_BESS_BoL** [float, optional]: Initial energy capacity of the centralized BESS. Default is 1000 kWh.
- **V_feeder** [float, optional]: Feeder voltage level. Default is 400 V.
- **dt** [float, optional]: Time resolution in hours. Default is 0.25.
- **V_ref_min** [float, optional]: Minimum voltage threshold. Default is 0.95×400 V.
- **V_ref_max** [float, optional]: Maximum voltage threshold. Default is 1.05×400 V.

3.28 Network_Control_Centralized()

This function performs centralized control of a battery energy storage system (BESS) within a low-voltage residential network. It evaluates the voltage profile and determines whether the BESS should charge or discharge based on voltage thresholds. The function updates power flows and battery states accordingly.

Arguments:

- **A** [array]: Admittance matrix of the network.
- **Z** [array]: Impedance matrix of the network.
- **I_0** [array]: Initial current vector.
- **V_0** [array]: Initial voltage vector.
- **selection** [list]: List of node indices with centralized BESS.
- **remaining** [list]: List of node indices without BESS.
- **df_P_L** [DataFrame]: Load profile per node.

- **df_P_PV_av [DataFrame]:** Available PV power per node.
- **df_P_PV [DataFrame]:** Actual PV power per node.
- **df_P_BESS [DataFrame]:** BESS power per node.
- **df_SOC_BESS [DataFrame]:** State of charge of BESS per node.
- **df_C_BESS [DataFrame]:** Capacity of BESS per node.
- **df_P_ref [DataFrame]:** Reference power setpoints per node.
- **timestep [int]:** Current time step in the simulation.
- **t_0 [int, optional]:** Starting time index. Default is 0.
- **Capacity_BESS_BoL [float, optional]:** Initial energy capacity of the centralized BESS. Default is 1000 kWh.
- **V_ref_min [float, optional]:** Minimum voltage threshold. Default is 0.95×400 V.
- **V_ref_max [float, optional]:** Maximum voltage threshold. Default is 1.05×400 V.
- **P_BESS_max [float, optional]:** Maximum power capacity of the centralized BESS. Default is 1000 kW.

3.29 Central_BESS_Control()

This function determines the charging or discharging power setpoints for a centralized battery energy storage system (BESS) based on the current state of charge (SoC), available PV power, and grid power requirements. It ensures that the BESS operates within its physical constraints and updates its state accordingly.

Arguments:

- **SoC_0 [float]:** Initial state of charge of the BESS (0 to 1).
- **Capacity_BESS [float]:** Current energy capacity of the BESS in kWh.
- **P_Grid_sum [float]:** Total power demand from the grid at the current time step.
- **P_PV_av [float]:** Available PV power at the current time step.
- **operation [str, optional]:** Operation mode: "Charge" or "Support". Default is "Charge".
- **charge_time [int, optional]:** Time (in time steps) over which to charge the BESS. Default is 15.
- **P_BESS_max [float, optional]:** Maximum power capacity of the BESS in kW. Default is 1000.
- **Capacity_BESS_BoL [float, optional]:** Initial energy capacity of the BESS at beginning of life. Default is 1000 kWh.

3.30 Voltage_Power_correlation()

This function analyzes the correlation between grid power and node voltage levels in a low-voltage distribution network. It generates plots of power, voltage, and energy prices over time, and fits a second-order polynomial model to describe the relationship between average grid power and minimum/maximum node voltages.

Arguments:

- **all_nodes** [list]: List of node indices to include in the analysis.
- **df_P_Grid** [DataFrame]: Grid power per node over time.
- **df_V_n** [DataFrame]: Voltage magnitude per node over time.
- **df_Prices** [DataFrame]: Energy price statistics over time.
- **V_feeder** [float, optional]: Nominal feeder voltage. Default is 400 V.
- **t0** [int, optional]: Start time index for plotting. Default is 0.
- **tf** [int, optional]: End time index for plotting. Default is 96 (1 day at 15-minute resolution).

3.31 Unify_array()

This utility function flattens a 2D array (typically a NumPy array or list of lists) into a single 1D list. It is used to prepare data for plotting or statistical analysis.

Arguments:

- **arr** [array-like]: A 2D array or list of lists to be flattened.

3.32 Make_voltage_boxplots()

This function generates boxplots of voltage distributions for different simulation cases or control strategies. It is typically used to compare voltage behavior across multiple scenarios in a low-voltage distribution network.

Arguments:

- **data** [list of arrays]: A list where each element is a 2D array representing voltage values over time for each node in a specific case.
- **headers** [list of str]: Labels for each case to be shown on the x-axis of the boxplot.
- **V_feeder** [float, optional]: Nominal feeder voltage used for per-unit normalization. Default is 400 V.

3.33 count_V_compliance()

This function evaluates voltage compliance in a set of simulation cases by counting the number or percentage of voltage values that fall outside a specified tolerance band around the nominal feeder voltage.

Arguments:

- **Case_n [list]:** A list of simulation results, where each element contains voltage data for a specific case.
- **lim [float, optional]:** Voltage tolerance limit in per unit (p.u.). Default is 0.05 (i.e., $\pm 5\%$).
- **percentage [bool, optional]:** If True, returns the percentage of violations. If False, returns the absolute count. Default is True.
- **index [int, optional]:** Index in the case data structure where voltage data is stored. Default is 4.
- **V_feeder [float, optional]:** Nominal feeder voltage in volts. Default is 400 V.

3.34 Plot_grid_behaviour()

This function visualizes the behavior of the electrical grid for a specific node across two simulation cases: a decentralized case and a centralized BESS control case. It plots voltage profiles, power flows, and battery state of charge over time.

Arguments:

- **case_n [list]:** List of simulation results for the decentralized case.
- **case_centralized [list]:** List of simulation results for the centralized BESS case.
- **node [int, optional]:** Node index to visualize. Default is 17.
- **i [int, optional]:** Index of the case to visualize. Default is -1 (last case).
- **start [int, optional]:** Start time index for plotting. Default is 0.
- **end [int, optional]:** End time index for plotting. Default is 1 (1 day at 15-minute resolution).
- **penetrations [list, optional]:** List of penetration levels (e.g., PV or BESS penetration). Default is `[10*i for i in range(1,10)]`.
- **V_feeder [float, optional]:** Nominal feeder voltage. Default is 400 V.

3.35 case_parameters()

This function returns a predefined set of simulation parameters based on a selected case index. Each case represents a different configuration of household energy systems, including the presence or absence of heat pumps (HP), battery energy storage systems (BESS), and thermal energy storage systems (TESS), as well as whether the system follows external control signals.

Arguments:

- **case [int]:** Index of the simulation case. Supported values range from 0 to 7.

3.36 TESS_SoC()

This function computes the state of charge (SoC) of a thermal energy storage system (TESS) based on its current temperature. The SoC is expressed as a percentage relative to the minimum and maximum operating temperatures.

Arguments:

- **T_TESS [float]:** Current temperature of the TESS in kelvin.
- **T_min [float, optional]:** Minimum operating temperature of the TESS in kelvin. Default is 323.15 K (50°C).
- **T_max [float, optional]:** Maximum operating temperature of the TESS in kelvin. Default is 363.15 K (90°C).

3.37 Plot_house_behaviour()

This function visualizes the electrical and thermal behavior of a single household node over a simulation period. It plots power flows, state of charge (SoC), temperatures, and energy prices, providing a comprehensive view of the household's energy dynamics.

Arguments:

- **case_n [list]:** List of simulation results for different cases.
- **node [int]:** Node index to visualize.
- **penetration [int, optional]:** Penetration level (e.g., PV or BESS) to select the case. Default is 20.
- **case [int, optional]:** Case index within the penetration level. Default is 0.
- **i [int, optional]:** Direct index of the case in the list. Overrides **penetration** and **case** if not -1. Default is -1.
- **start [int, optional]:** Start time index for plotting. Default is 0.
- **end [int, optional]:** End time index for plotting. Default is 672 (7 days at 15-minute resolution).
- **penetrations [list, optional]:** List of penetration levels. Default is [10*i for i in range(1,10)].

3.38 Cost_function_1()

This function defines a quadratic cost function used in optimization problems for power setpoint allocation. It calculates the sum of squared deviations between optimized power values and reference power values for each node.

Arguments:

- **x [list or array]:** Decision variables representing the optimized power setpoints for each node.
- ***args:** Additional arguments passed to the function. The 6th element (index 5) is expected to be the reference power values S_{n0} .

3.39 Cost_function_2()

This function defines a voltage deviation cost function used in optimization problems for power setpoint allocation. It calculates the sum of squared deviations between the estimated node voltages and the nominal feeder voltage, based on the current power injections.

Arguments:

- **x [list or array]:** Decision variables representing the optimized power setpoints for each node.
- ***args:** Additional arguments passed to the function, expected in the following order:
 - **A [array]:** Admittance matrix of the network.
 - **B [array]:** Vector with 1 at the feeder node and 0 elsewhere.
 - **Z [array]:** Impedance matrix of the network.
 - **V0 [array]:** Initial voltage vector.
 - **I0 [array]:** Initial current vector.

3.40 Cost_function()

This function defines a weighted combination of two cost functions used in optimization problems for power setpoint allocation in a distribution network. It combines the deviation from reference power values and the deviation from nominal voltage levels.

Arguments:

- **x [list or array]:** Decision variables representing the optimized power setpoints for each node.
- ***args:** Additional arguments passed to the function, expected in the following order:
 - **A [array]:** Admittance matrix of the network.
 - **B [array]:** Vector with 1 at the feeder node and 0 elsewhere.
 - **Z [array]:** Impedance matrix of the network.
 - **V0 [array]:** Initial voltage vector.
 - **I0 [array]:** Initial current vector.
 - **Sn0 [list or array]:** Reference power values for each node.
 - **weights [list of two floats]:** Weights for the two cost components: [weight_Sn, weight_V].

3.41 Find_power_setpoints()

This function solves an optimization problem to determine the optimal power setpoints for each node in a distribution network. It minimizes a cost function subject to voltage and power constraints, using the Sequential Least Squares Programming (SLSQP) method.

Arguments:

- **A [array]:** Admittance matrix of the network.

- **Z [array]:** Impedance matrix of the network.
- **B [array]:** Vector with 1 at the feeder node and 0 elsewhere.
- **V0 [array]:** Initial voltage vector.
- **I0 [array]:** Initial current vector.
- **Sn0 [list or array]:** Reference power values for each node.
- **all_nodes [list]:** List of all node indices in the network.
- **selected_nodes [list]:** List of nodes for which power setpoints are to be optimized.
- **pu_lim [float, optional]:** Voltage tolerance limit in per unit. Default is 0.05.
- **v_feeder [float, optional]:** Nominal feeder voltage. Default is 400 V.
- **weights [list of two floats, optional]:** Weights for the cost components: deviation from reference power and voltage deviation. Default is [1162690, 13].
- **plotting [bool, optional]:** If True, plots the optimized power and voltage profiles. Default is False.

Chapter 4

Example_CIGRE_18

This example script demonstrates the use of the `Network_builder_public` library to simulate and analyze the operation of a low-voltage residential distribution network based on the CIGRE benchmark. It evaluates different energy management strategies and their impact on voltage regulation, grid interaction, and distributed energy resource (DER) coordination. The example file can be found in the repository [5].

Purpose

The script simulates three scenarios:

1. **Base Case:** Uncontrolled operation with PV and heat pumps.
2. **Aggregation Scenario:** Coordinated control of distributed BESS and thermal storage.
3. **Centralized BESS:** A single large BESS unit at the feeder end.

Usage

To run the example:

1. Ensure the following input files are available:
 - `Cigre_18.2.xlsx` – Network topology and house types.
 - `Wires.xlsx` – Cable impedance data.
 - `S_n_CIGRE_18.csv` – Load profiles.
 - `Tamb_15min.csv`, `Radiation_1min.csv`, `PV_15min.csv` – Environmental and PV data.
2. Set the desired `season` (e.g., 'Winter').
3. Execute the script in a Python environment with required dependencies.

Key Variables

- `DF_Network` – DataFrame containing the network structure.
- `S_n` – Load matrix per node.

- **selected_nodes** – Nodes with controllable DERs.
- **control_type** – Control strategy (e.g., 'heuristic').
- **Case_CIGRE**, **Case_CIGRE_controlled**, **Case_CIGRE_centralized** – Scenario results.

Outputs

The script generates:

- **Voltage boxplots** comparing voltage profiles across scenarios.
- **Compliance statistics** showing voltage violations.
- **Correlation plots** between grid power and voltage.
- **House-level plots** showing power flows, SoC, and thermal behavior.
- **Grid-level plots** showing voltage and power behavior at the feeder.

Bibliography

- [1] J. Alpízar-Castillo, “Residential multi-carrier energy storage systems as potential flexibility providers in low-voltage networks: A new player has joined the game,” Ph.D. dissertation, Delft University of Technology, 2025. [Online]. Available: <https://repository.tudelft.nl/record/uuid:76075049-7656-4c49-b206-e742a65b6062>.
- [2] J. Alpízar-Castillo, “Model predictive control implementation for the ocean grazer wave energy converter with a port-hamiltonian model,” M.S. thesis, University of Groningen, Costa Rica Institute of Technology, 2018. [Online]. Available: <https://repositoriotec.tec.ac.cr/handle/2238/10439>.
- [3] J. Alpízar-Castillo, *CSV Python Tools*, https://github.com/jjac13/CSV_Python_Tools, 2022.
- [4] J. Alpízar-Castillo, L. Ramírez-Elizondo, A. van Voorden, and P. Bauer, “Aggregated residential multi-carrier energy storage as voltage control provider in low-voltage distribution networks,” *Journal of Energy Storage*, vol. 132, p. 117507, 2025, ISSN: 2352-152X. DOI: <https://doi.org/10.1016/j.est.2025.117507>. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S2352152X25022200>.
- [5] J. Alpízar-Castillo, *LV network simulator public*, https://github.com/jjac13/LV_network_simulator_public, 2025.